

## Projeto MenuFlex – Atividade Somativa

### Contextualização

A empresa MenuFlex Tecnologia presta serviços de desenvolvimento de sistemas web para pequenos negócios. Um de seus clientes, o restaurante “Cantina Bella Vita”, solicitou a criação de um sistema para gerenciamento digital de seu cardápio.

A empresa já decidiu que o sistema será desenvolvido em arquitetura web, utilizando API REST para o back-end.

Antes do início do desenvolvimento completo, a equipe técnica precisa validar a modelagem, as decisões tecnológicas e a estrutura da API, garantindo que o projeto seja escalável e organizado.

Você foi designado para realizar essa análise técnica inicial.

### Etapas A – Modelagem Orientada a Objetos (Diagrama de Classe)

1 – Elabore o **diagrama de classe** da entidade **Produto**, considerando um sistema de cardápio digital.

O diagrama deve conter, no mínimo:

- Atributos e métodos essenciais;
- Tipos de dados;



## **Etapas B – Persistência e Base de Dados**

1 – Explique como a entidade **Produto** seria armazenada em um banco de dados.

Sua resposta deve abordar:

- Tipo de banco de dados mais adequado (relacional ou não relacional);
- Justificativa da escolha;
- Forma de interação entre a API e o banco de dados.

O banco de dados mais adequado para a entidade é o relacional, pois armazena de forma organizada e mais estruturada as informações que serão retidas dentro da entidade. Dessa forma, a API terá uma interação com o banco de dados de modo a criar uma piscina de conexões, fazendo com que as informações retidas dentro da API passem diretamente para o banco de dados.

2 – Veja o arquivo database.js:

```
const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'bellavita',
  porta: 3306
})
```

Por que essa conexão não vai funcionar? Explique qual é o erro e como corrigir.

Seguindo a estruturação inicial que nos foi ensinada, a qual não é idêntica a demonstrada no exemplo, acredito que o erro esteja na exibição de valores que deveriam ser secretos, e, nesse caso enviados ao arquivo .env para reter as informações protegidas, e fazer com que esse arquivo [database.js](#) apenas mencionasse as informações presentes no .env mas de maneira “subjativa”, de modo a proteger os dados importantes e confidenciais se uma API.

3 - Observe o código:

```
app.put('/produtos/:id', async (req, res) => {  
  const { id } = req.params  
  const dados = req.body  
  
  await queryAsync("UPDATE produto SET ? WHERE id = id", [dados])  
  res.send("Atualizado")  
})
```

Por que esse UPDATE não irá atualizar nenhum produto?

O const de dados como req.body não define especificamente quais serão os dados a serem alterados. Nesse caso, seria necessário listar dentro das chaves todos os atributos da entidade produto que poderiam ser atualizados. Além disso, o código não possui validação de informações, nem mesmo um espaço para atualizar a informação antiga pela nova. Seria necessária a criação de uma variável que reteria as novas informações e uma validação para garantir que o novo valor não é nulo (não havendo, assim, nada passível de atualização) e que ele preenche os requisitos necessários para determinado atributo (como tipo ou valor). Ademais, a estruturação da queryAsync está incorreta, já que seria necessário inserir como dados para substituir o endpoint "?" o id do produto em questão e, ao invés de "dados", a variável que reteria as novas informações atualizadas.

### **Etapas C – Arquitetura da API REST**

1 - Descreva a estrutura básica de uma **API REST** para gerenciamento de produtos.

Inclua em sua resposta:

- Endpoints necessários para as operações de CRUD;
- Métodos HTTP associados a cada operação;
- Organização lógica das rotas.

Uma API REST para gerenciamento de produtos precisa de todas as operações do CRUD que podem ser implementadas para a manipulação dos atributos da entidade. Nesse caso, seriam usados métodos HTTP como GET, POST, PUT e DELETE, que, de maneira lógica, permitem selecionar todos os elementos da entidade, selecioná-los especificamente pelo ID, criar um novo elemento, atualizar um elemento já existente a partir do ID e deletar um elemento utilizando novamente o ID. Dessa forma, é possível manipular todas as informações da API de maneira a ligá-las diretamente ao banco de dados.

2 - Analise a rota abaixo:

```
app.get('/produtos/id', async (req, res) => {  
  const id = req.params.id  
  const produto = await queryAsync("SELECT * FROM produto WHERE id = ?", [id])  
  res.json(produto)  
})
```

Por que essa rota nunca vai funcionar ao buscar um produto pelo ID?

Quando criamos a const id, esperamos que o req.params encontre esse dado sem precisar relacioná-lo ao .id como feito na rota. Além disso, é necessário que na criação do caminho '/produtos/id' haja um ":" para demonstrar ao código que esse valor faz parte de um dos atributos da entidade, e que será substituído na URL quando o usuário buscar pelo id que quer. A maneira correta seria '/produtos/:id'. Destarte, é recomendado utilizar um sistema de trycatch para que todos os possíveis erros sejam validados, além de fazer com que a devolutiva (res.json(produto)) seja mais completa e elaborada.

3 – Desenvolva o método DELETE de produtos.

```
// 5) DELETAR PRODUTO
// Gabriela Carnevali Gonçalves Lima |
app.delete ('/produtos/:id', async (req, res) => {
  try {
    const { id } = req.params

    if (!id || isNaN(id)) {
      return res.status(400).json({
        sucesso: false,
        mensagem: "ID inválido"
      })
    }

    const existe = await queryAsync("SELECT * FROM produto WHERE id = id", [id])
    if (existe.length === 0) {
      return res.status(404).json({
        sucesso: false,
        mensagem: "Produto não encontrado"
      })
    }

    await queryAsync("DELETE FROM produto WHERE id = ?", [id])

    res.json({
      sucesso: true,
      mensagem: "Produto deletado com sucesso"
    })
  } catch (erro) {
    res.status(500).json({
      sucesso: false,
      mensagem: "Erro ao deletar produto",
      erro: erro.message
    })
  }
})
```

## Etapa D – Frameworks e Ambiente de Desenvolvimento

1 - Explique por que o uso do **Node.js com Express** é adequado para o desenvolvimento dessa API.

Sua resposta deve considerar:

- Características do Node.js;
- Papel do framework Express;
- Benefícios para aplicações web.

O [Node.js](#) possibilita a conexão facilitada da API com o banco de dados e com as operações necessárias para a criação de uma aplicação de qualidade. O framework Express faz com que seja possível a criação de métodos HTTP para a criação do CRUD, além de ambas as ferramentas melhorarem a qualidade das aplicações WEB.

2 – Descreva, de forma clara e organizada, as etapas iniciais para criar um projeto web em Node.js com persistência de dados em MySQL. Explique também como você estruturaria a organização do projeto (pastas e arquivos principais).

A primeira etapa é criar uma pasta para a API no terminal, utilizando mkdir. Depois, utilizando o comando `npm init -y` e `npm install express dotenv mysql2`, é possível criar os arquivos `package.json` e `package-lock.json`, que são responsáveis por reter as informações e componentes dos frameworks e bibliotecas. Depois disso, cria-se a pasta `src`, que conterá dentro dela a pasta `config` com [database.js](#) e os arquivos [app.js](#) e [server.js](#). O arquivo `.env` fica fora do `src`, para não ser enviado ao github. Após esse processo e a criação dos códigos em `.env`, [server.js](#) e [database.js](#), utilizamos o comando `node -watch ./src/server.js`, para garantir que a conexão com o banco de dados foi efetiva e em qual porta o servidor está rodando.

3 – Para que serve o arquivo `.env` e por que não devemos subi-lo no github?

O arquivo `.env` é usado para reter informações confidenciais do projeto, ou seja, senhas, credenciais, informações de localização, porta em que está rodando, nome do banco de dados, entre outros. Esse arquivo não pode ser enviado ao github pois contém informações que, se caírem em mãos erradas, podem comprometer a funcionalidade do sistema, já que permitem acesso direto a informações que permitem que a API rode.